

NLP 期末報告

基於深度學習之桌球戰術與結果
預測系統

AI CUP 2026

小組成員

112590059

資工三
傅啓碩

專題介紹

本專題以 AI CUP 2026 春季賽「基於時序資料之桌球戰術與結果預測競賽」為主題，利用桌球比賽中的時序資料建立深度學習模型，預測球員戰術與比賽結果。

由於桌球擊球行為具有高度時間關聯性，因此需要使用時序模型進行分析。此外，資料中包含多種球員與比賽特徵，可能面臨序列長度不同、類別資料多以及資料分布不平均等問題。

資料介紹

features	說明
rally_uid	小分的唯一識別碼
sex	比賽性別（男(1) / 女(2)）
match	比賽的唯一識別碼
numberGame	小局數（第幾局）
rally_id	小局內的小分編號
strikeNumber	小分內的揮拍次序

資料介紹

features	說明
scoreSelf	主視角選手的得分
scoreOther	對側選手的得分
serverGetPoint	發球者是否得分（1=是, 0=否）
gamePlayerId	主視角選手的 ID
gamePlayerOtherId	對側視角選手的 ID
strikeId	揮拍狀態或動作類型

資料介紹

features	說明
handId	正手或反手揮拍
strengthId	擊球力道
spinId	球的旋轉方式
pointId	球的落點位置
actionId	擊球方式
positionId	球員站位區域

資料介紹

rally_uid, sex, match, numberGame, rally_id, strikeNumber, scoreSelf, scoreOther, serverGetPoint, gamePlayerId, gamePlayerOtherId, strikeId, handId, strengthId, spinId, pointId, actionId, positionId

```
1,2,1,1,1,12,0,0,0,2,1,4,2,1,2,4,10,0
1,2,1,1,1,13,0,0,0,1,2,4,2,1,2,9,10,0
1,2,1,1,1,14,0,0,0,2,1,4,1,3,3,9,3,0
1,2,1,1,1,15,0,0,0,1,2,4,0,0,0,0,0,0
2,2,1,1,2,1,0,1,1,1,2,1,1,2,3,9,15,1
2,2,1,1,2,2,1,0,1,2,1,2,2,1,2,8,12,1
2,2,1,1,2,3,0,1,1,1,2,4,1,2,2,7,3,0
2,2,1,1,2,4,1,0,1,2,1,4,2,0,0,0,13,0
3,2,1,1,3,1,1,1,0,2,1,1,2,3,4,9,15,1
3,2,1,1,3,2,1,1,0,1,2,2,2,2,3,9,6,1
3,2,1,1,3,3,1,1,0,2,1,4,2,1,2,3,12,0
```

baseline code - 訓練前準備

定義資料

```
FEATURES = ["sex", "handId", ..., "strikeNumber"]
```

```
train = pd.read_csv(args.train).sort_values(["rally_uid", "strikeNumber"])  
test  = pd.read_csv(args.test).sort_values(["rally_uid", "strikeNumber"])
```

將欄位轉成 index，並保留0當 padding

```
cats = {c: pd.Categorical(train[c]).categories for c in FEATURES}  
def encode_frame(df):  
    outs = []  
    for col in FEATURES:  
        codes = pd.Categorical(df[col], categories=cats[col]).codes + 1  
        outs.append(np.asarray(codes, dtype=np.int64))  
    return np.stack(outs, axis=1)
```

baseline code - 訓練前準備

```
class RallyDataset(Dataset):
    def __init__(self, X, yA, yP, yR, L):
        self.X = torch.tensor(X, dtype=torch.long)
        self.yA = torch.tensor(yA, dtype=torch.long)
        self.yP = torch.tensor(yP, dtype=torch.long)
        self.yR = torch.tensor(yR, dtype=torch.float32)
        self.L = torch.tensor(L, dtype=torch.long)
    def __len__(self): return self.X.shape[0]
    def __getitem__(self, i): return self.X[i], self.yA[i], self.yP[i], self.yR[i],
```

建立Dataset的結構

baseline code - 訓練前準備

```
X_list, yA_list, yP_list, yR_list, L_list = [], [], [], [], []
for rid, g in train.groupby("rally_uid"):
    if len(g) < 2: continue
    X = encode_frame(g)[: -1]
    yA = g["actionId"].values[1:].astype(np.int64)
    yP = g["pointId"].values[1:].astype(np.int64)
    X_list.append(X); yA_list.append(yA); yP_list.append(yP)
    yR_list.append(int(g["serverGetPoint"].iloc[0])); L_list.append(len(X))
```

```
MAXLEN = max(L_list)
X_all = np.stack([pad2d(s, MAXLEN) for s in X_list])
yA_all = np.stack([pad1d(s, MAXLEN) for s in yA_list])
yP_all = np.stack([pad1d(s, MAXLEN) for s in yP_list])
yR_all = np.array(yR_list, dtype=np.float32)
L_all = np.array(L_list, dtype=np.int64)
```

建立Dataset

baseline code - 訓練前準備

切割測試集和驗證集

```
idx = np.arange(len(X_all))
tr_idx, va_idx = train_test_split(idx, test_size=args.val_size,
    random_state=42, stratify=(yR_all>0.5))
X_tr, X_va = X_all[tr_idx], X_all[va_idx]
yA_tr, yA_va = yA_all[tr_idx], yA_all[va_idx]
yP_tr, yP_va = yP_all[tr_idx], yP_all[va_idx]
yR_tr, yR_va = yR_all[tr_idx], yR_all[va_idx]
L_tr, L_va = L_all[tr_idx], L_all[va_idx]

train_ds = RallyDataset(X_tr, yA_tr, yP_tr, yR_tr, L_tr)
val_ds = RallyDataset(X_va, yA_va, yP_va, yR_va, L_va)
train_loader = DataLoader(train_ds, batch_size=args.batch, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=max(args.batch*2,128), shuffle=False)
```

baseline code - 訓練前準備

建立 loss 和 model , 準備訓練

建立 loss 和 model，準備訓練

```
act_counts = np.bincount(yA_tr[yA_tr!=-1].ravel(), minlength=n_act) + 1
pt_counts  = np.bincount(yP_tr[yP_tr!=-1].ravel(), minlength=n_pt) + 1
act_w = torch.tensor(1.0/act_counts, dtype=torch.float32); act_w = (act_w * (n_act/act_w.sum()))
pt_w  = torch.tensor(1.0/pt_counts,  dtype=torch.float32); pt_w  = (pt_w  * (n_pt /pt_w.sum()))
num_tokens_per_feature = [len(cats[c]) + 1 for c in FEATURES]

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = MultiTaskLSTM(num_tokens_per_feature, n_act, n_pt, emb_dim=args.emb,
|         |         hidden=args.hidden, num_layers=args.layers, dropout=args.drop).to(device)
ce_action = nn.CrossEntropyLoss(ignore_index=-1, weight=act_w.to(device))
ce_point  = nn.CrossEntropyLoss(ignore_index=-1, weight=pt_w.to(device))
bce_rally = nn.BCEWithLogitsLoss()
opt = torch.optim.Adam(model.parameters(), lr=args.lr)
```

baseline code - 模型介紹

模型建置

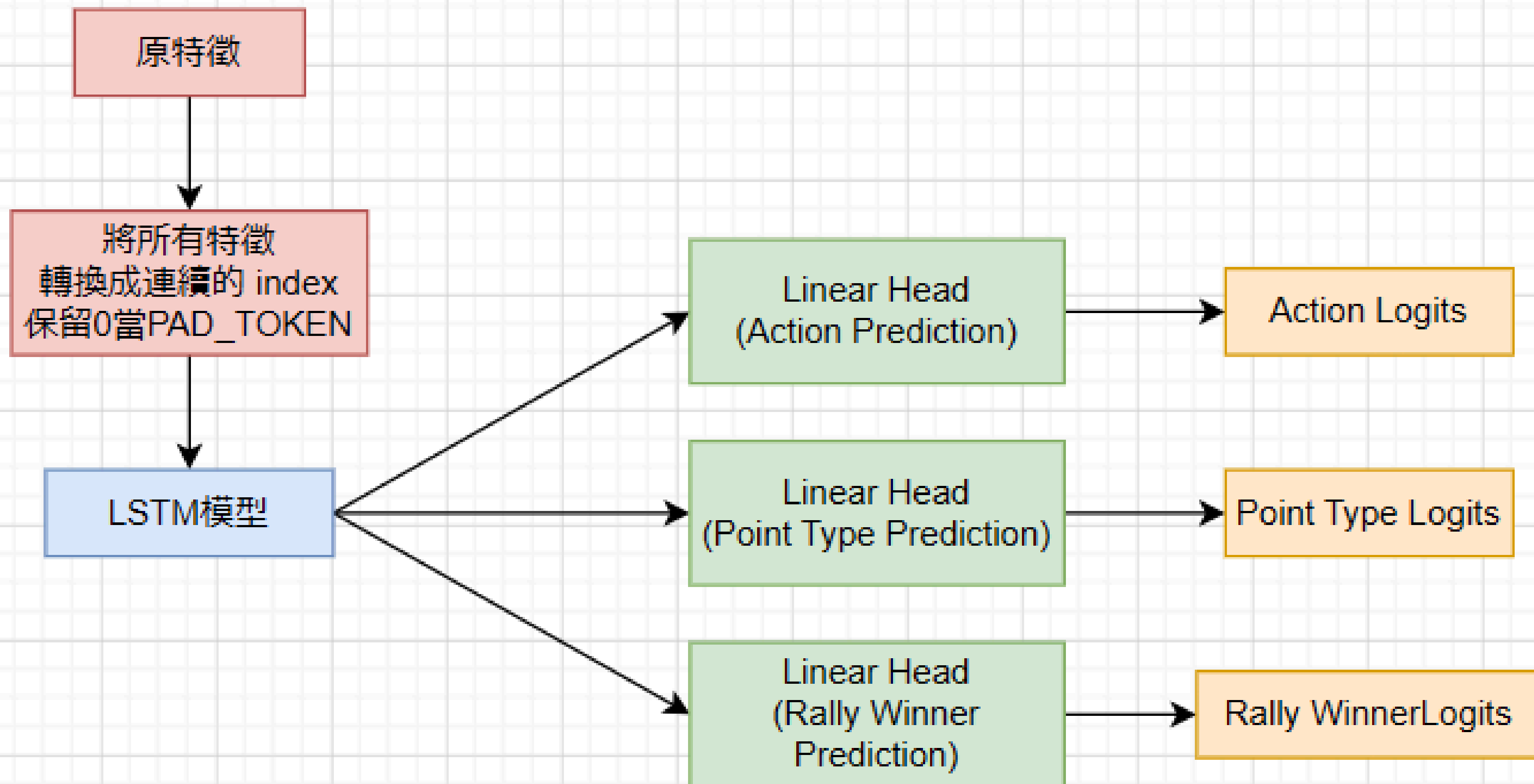
```
class MultiTaskLSTM(nn.Module):
    def __init__(self, num_tokens_per_feature, n_act, n_pt, emb_dim=16, hidden=128, num_layers=1):
        super().__init__()
        self.embs = nn.ModuleList([nn.Embedding(n+1, emb_dim, padding_idx=PAD_TOKEN)
                                     for n in num_tokens_per_feature])
        self.lstm = nn.LSTM(len(num_tokens_per_feature)*emb_dim, hidden,
                             num_layers=num_layers, batch_first=True,
                             dropout=dropout if num_layers>1 else 0.0, bidirectional=False)
        self.drop = nn.Dropout(dropout)
        self.act_head = nn.Linear(hidden, n_act)
        self.pt_head = nn.Linear(hidden, n_pt)
        self.rly_head = nn.Linear(hidden, 1)
```

baseline code - 模型介紹

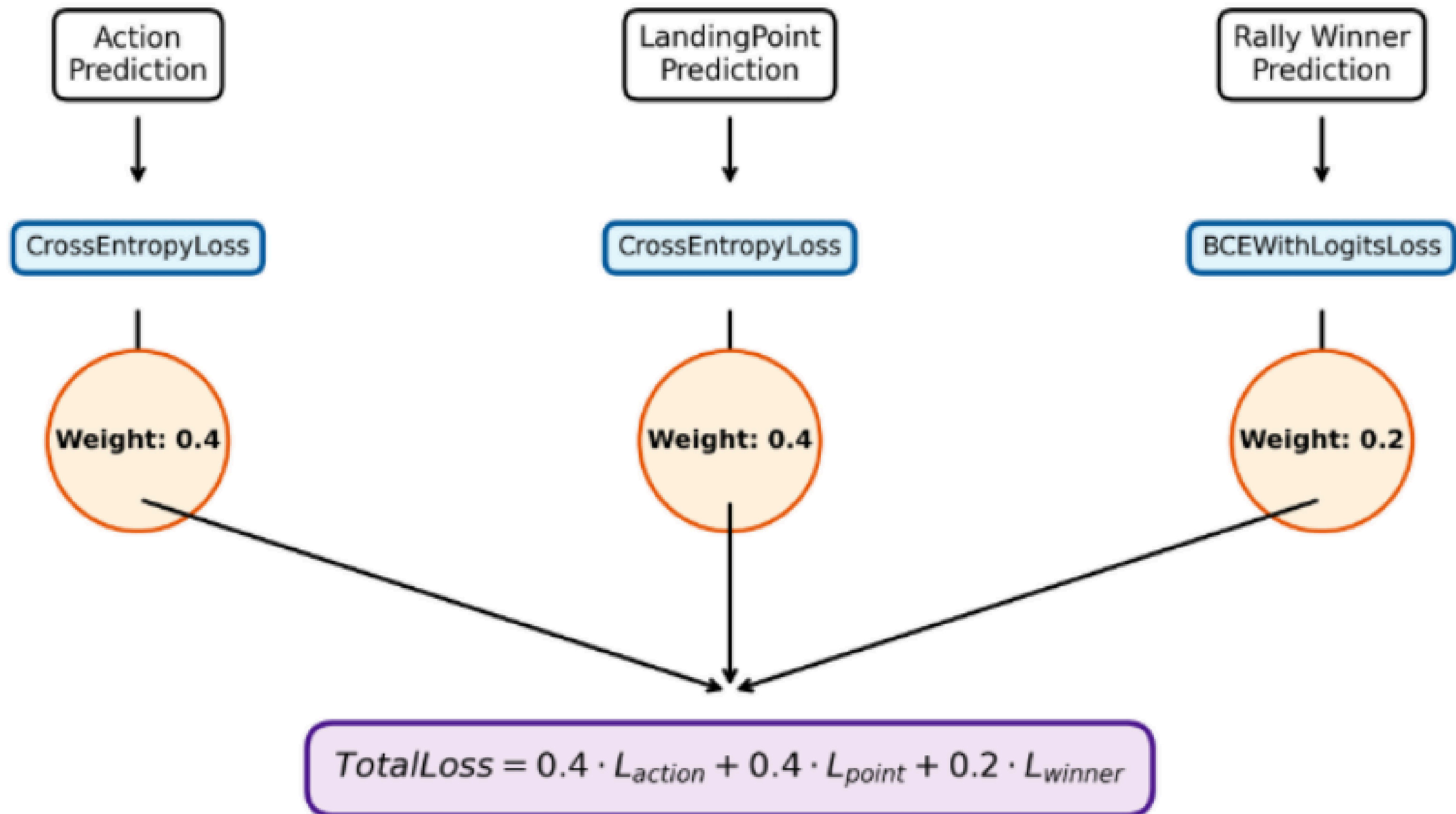
建立forward

```
def forward(self, X, lengths):
    es = [emb(X[:, :, i]) for i, emb in enumerate(self.embs)]
    x = torch.cat(es, dim=-1)
    packed = nn.utils.rnn.pack_padded_sequence(x, lengths.cpu(), batch_first=True, enforce_sorted=False)
    o, _ = self.lstm(packed)
    o, _ = nn.utils.rnn.pad_packed_sequence(o, batch_first=True, total_length=X.size(1))
    o = self.drop(o)
    mask = (X[:, :, 0] != PAD_TOKEN).float().unsqueeze(-1)
    denom = mask.sum(dim=1).clamp(min=1.0)
    mean_hidden = (o * mask).sum(dim=1) / denom
    return self.act_head(o), self.pt_head(o), self.rly_head(mean_hidden).squeeze(1)
```

baseline code - 模型介紹



baseline code - Loss Function



改動部分 - 特徵

加入一些統計以及組合特徵

- 分數差

```
data["scoreDiff"] = data["scoreSelf"] - data["scoreOther"]
```

- side 特徵

```
position_map = {0: "無", 1: "正手", 2: "中間", 3: "反手", 4: "正手",  
                5: "中間", 6: "反手", 7: "正手", 8: "中間", 9: "反手"}  
data['side'] = data["pointId"].map(position_map)
```

- 玩家對戰組合特徵

```
data['player_pk_key'] = np.maximum(data['gamePlayerId'], data['gamePlayerOtherId']).astype(str) \  
+ '_' + np.minimum(data['gamePlayerId'], data['gamePlayerOtherId']).astype(str)
```

- 球員勝率

```
data["p1_winrate"] = data["gamePlayerId"].map(self.train_player_winrate).fillna(0.5)  
data["p2_winrate"] = data["gamePlayerOtherId"].map(self.train_player_winrate).fillna(0.5)  
data["winrate_diff"] = data["p1_winrate"] - data["p2_winrate"]
```

改動部分 - 特徵

加入一些統計以及組合特徵

- 技能頻率特徵

```
data = self.make_freq_data(data, "action_freq")
data = self.make_freq_data(data, "spin_freq")
data = self.make_freq_data(data, "point_freq")
```

```
make_freq_data(self, data, freq_name):
    freq = getattr(self, freq_name, None)
    if freq is not None:
        for col in freq.columns:
            data[f"{freq_name}_player_{col}"] = data["gamePlayerId"].map(freq[col]).fillna(0)
            data[f"{freq_name}_other_player_{col}"] = data["gamePlayerOtherId"].map(freq[col]).fillna(0)
    return data
```

改動部分 - 特徵

加入一些統計以及組合特徵

```
FEATURES2 = ([
    "p1_winrate", "p2_winrate", "winrate_diff"]
+ [f'action_freq_player_{i}' for i in range(19)]
+ [f'action_freq_other_player_{i}' for i in range(19)]
+ [f'spin_freq_player_{i}' for i in range(6)]
+ [f'spin_freq_other_player_{i}' for i in range(6)]
+ [f'point_freq_player_{i}' for i in range(10)]
+ [f'point_freq_other_player_{i}' for i in range(10)]
)
```

FEATURE2 會直接接在 embedding 後面

改動部分 - 模型架構

- Attention Pool實作

```
def pool(o, query, mask):  
    score = torch.matmul(o, query) # [B,T]  
    score = score.masked_fill(~mask, -1e9)  
    weight = torch.softmax(score, dim=1)  
    return (weight.unsqueeze(-1) * o).sum(1)
```



改動部分 - 模型架構

每個 task 有自己的 query

```
self.act_query = nn.Parameter(torch.randn(hidden))  
self.pt_query  = nn.Parameter(torch.randn(hidden))  
self.rly_query = nn.Parameter(torch.randn(hidden))
```

```
mask = (torch.arange(T, device=device)[None] < lengths[:,None])
```

```
act_o = pool(o, self.act_query, mask)
```

```
pt_o  = pool(o, self.pt_query, mask)
```

```
rly_o = pool(o, self.rly_query, mask)
```

```
la, lp, lr = self.act_head(act_o), self.pt_head(pt_o), self.rly_head(rly_o).squeeze(-1)
```

改動部分 - Inference

Inference 改成只看最後 k 步

```
for rid, g in data.groupby("rally_uid", sort=False):
    g = g.reset_index(drop=True)
    Xg, valueXg = encode_frame(g, FEATURES, FEATURES2, cats)

    hist = Xg[max(0, len(Xg) - k):]    # 最後 k 筆
    Xp, T = pad2d_to_k(hist, k)
    hist2 = valueXg[max(0, len(valueXg) - k):]    # 最後 k 筆
    valueXp, T2 = pad2d_to_k_float(hist2, k)
    assert T==T2

    self.X.append(Xp)
    self.valueX.append(valueXp)
    self.L.append(T)
    self.rid.append(int(rid))
```

分數計算

1. 任務一：下一拍（第 n 拍）的球種預測（參數名稱：actionId）

評估指標 (S_1)：Macro F1-Score

2. 任務二：下一拍（第 n 拍）的落點 (以九宮格分隔球桌區域) 預測（參數名稱：pointId）

評估指標 (S_2)：Macro F1-Score

3. 任務三：根據此回合（Rally）已進行的擊球序列，預測最終此Rally的發球者是否得分。（參數名稱：serverGetPoint）

評估指標 (S_3)：AUC-ROC (Area Under the ROC Curve)

w_1, w_2, w_3 訂為：0.4, 0.4 和 0.2

$$Score = w_1 \times S_1 + w_2 \times S_2 + w_3 \times S_3$$

預測結果

Public Leaderboard		Private Leaderboard	
檔名	上傳時間	評估結果	排名
submission_lstm_...csv 112590059	2026-06-02 23:22:39	0.3839692	82/423

Public Leaderboard		Private Leaderboard	
檔名	上傳時間	評估結果	排名
submission_lstm_...csv 112590059	2026-06-02 23:22:39	0.2527775	381/423

可能原因

- private和目前模型資料分布不相同
每局長度，action和point type和rally winner分布
- 資料主視角影響判斷
資料每個 Timestamp 主視角相關參數都會掉換
- 太過趨近public
可能 private出現許多未出現 playerId，或 public 有許多相關場數，導致 public 分數較高。



THANK YOU